SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1402 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code (BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 30 **Total Marks:60**

Annexure A

MOCK INTERVIEW

Code of Conduct:

I _Monika.R(192324281)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Annexure A

MOCK INTERVIEW

QUESTIONS

1. What is a compiler?

2. What are the different phases of a compiler?

3. What is lexical analysis?

4. What is the role of a lexical analyzer?

5. What is a token? Give an example.

6. What is a lexeme?

7. What is the difference between token and lexeme?

8. What is a pattern in lexical analysis?

9. What are keywords and identifiers?

10. What are the common errors detected by a lexical analyzer?

11. What is syntax analysis?

12. What is the role of a parser?

13. What is a parse tree?

14. What is the difference between parse tree and syntax tree?

15. What is ambiguity in grammar?

16. What is left recursion?

17. Why should left recursion be removed?

18. What is left factoring?

19. What is predictive parsing?

20. What is the dangling else problem?

21. What is FIRST set?

22. What is FOLLOW set?

23. Why are FIRST and FOLLOW sets important?

24. What is LL(1) grammar?

25. What is an LL(1) parsing table?

26. When does a conflict occur in LL(1) parsing?

27. Why is ambiguous grammar not suitable for LL(1)?

28. How does a predictive parser work?

29. What is bottom-up parsing?

30. What is shift-reduce parsing?

ANSWERS

1. What is a compiler?

A compiler is a system software that translates a high-level programming language into machine or target code. It also detects and reports errors in the program.

2. What are the different phases of a compiler?

The main phases are lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and code generation.

3. What is lexical analysis?

Lexical analysis is the first phase of a compiler. It converts the source program into tokens such as keywords, identifiers, operators, and constants.

4. What is the role of a lexical analyzer?

A lexical analyzer reads the source code, identifies tokens, removes spaces/comments, and reports lexical errors.

5. What is a token? Give an example.

A token is a meaningful unit of a program recognized by the compiler. Examples are if, sum, +, and 10.

6. What is a lexeme?

A lexeme is the actual sequence of characters that forms a token. For example, in int count, count is a lexeme of the identifier token.

7. What is the difference between token and lexeme?

A token represents the category, while a lexeme represents the actual value. Example: total is a lexeme and identifier is its token.

8. What is a pattern in lexical analysis?

A pattern is a rule used to identify a token. For example, an identifier can be represented as a letter followed by letters or digits.

9. What are keywords and identifiers?

Keywords are reserved words with special meaning, such as int, if, and while. Identifiers are names given by programmers, such as sum and count.

10. What are the common errors detected by a lexical analyzer?

It detects invalid characters, invalid identifiers, unterminated strings, and invalid numbers.

11. What is syntax analysis?

Syntax analysis checks whether the sequence of tokens follows the grammar rules of the programming language.

12. What is the role of a parser?

A parser analyzes the syntax of the input and constructs a parse tree or syntax tree.

13. What is a parse tree?

A parse tree is a tree representation of the syntactic structure of a program according to its grammar.

14. What is the difference between parse tree and syntax tree?

A parse tree contains all grammar details, while a syntax tree is a simplified representation containing only important program constructs.

15. What is ambiguity in grammar?

A grammar is ambiguous when one input string can have more than one parse tree or derivation.

16. What is left recursion?

Left recursion occurs when a non-terminal appears as the first symbol on the right side of its own production. Example: $E \rightarrow E + T$.

17. Why should left recursion be removed?

Left recursion should be removed because it can cause infinite recursion in recursive descent and predictive parsers.

18. What is left factoring?

Left factoring is a grammar transformation that removes common prefixes from productions so that the parser can easily choose the correct production.

19. What is predictive parsing?

Predictive parsing is a top-down parsing technique that selects the correct production using the next input symbol, without backtracking.

20. What is the dangling else problem?

The dangling else problem occurs when it is unclear which if statement an else belongs to. It is resolved using grammar rules or associating else with the nearest unmatched if.

21. What is FIRST set?

FIRST contains the symbols that can appear first in strings derived from a grammar symbol.

22. What is FOLLOW set?

FOLLOW contains the symbols that can appear immediately after a non-terminal.

23. Why are FIRST and FOLLOW sets important?

They help in constructing predictive parsing tables and selecting the correct production.

24. What is LL(1) grammar?

An LL(1) grammar can be parsed left-to-right with one lookahead symbol without backtracking.

25. What is an LL(1) parsing table?

It is a table that tells the parser which production to use based on the non-terminal and lookahead symbol.

26. When does a conflict occur in LL(1) parsing?

A conflict occurs when more than one production can be selected for the same lookahead symbol.

27. Why is ambiguous grammar not suitable for LL(1)?

Because ambiguity allows multiple parsing choices, while LL(1) requires a single choice.

28. How does a predictive parser work?

It uses the current stack symbol and one lookahead symbol to select a production from the parsing table.

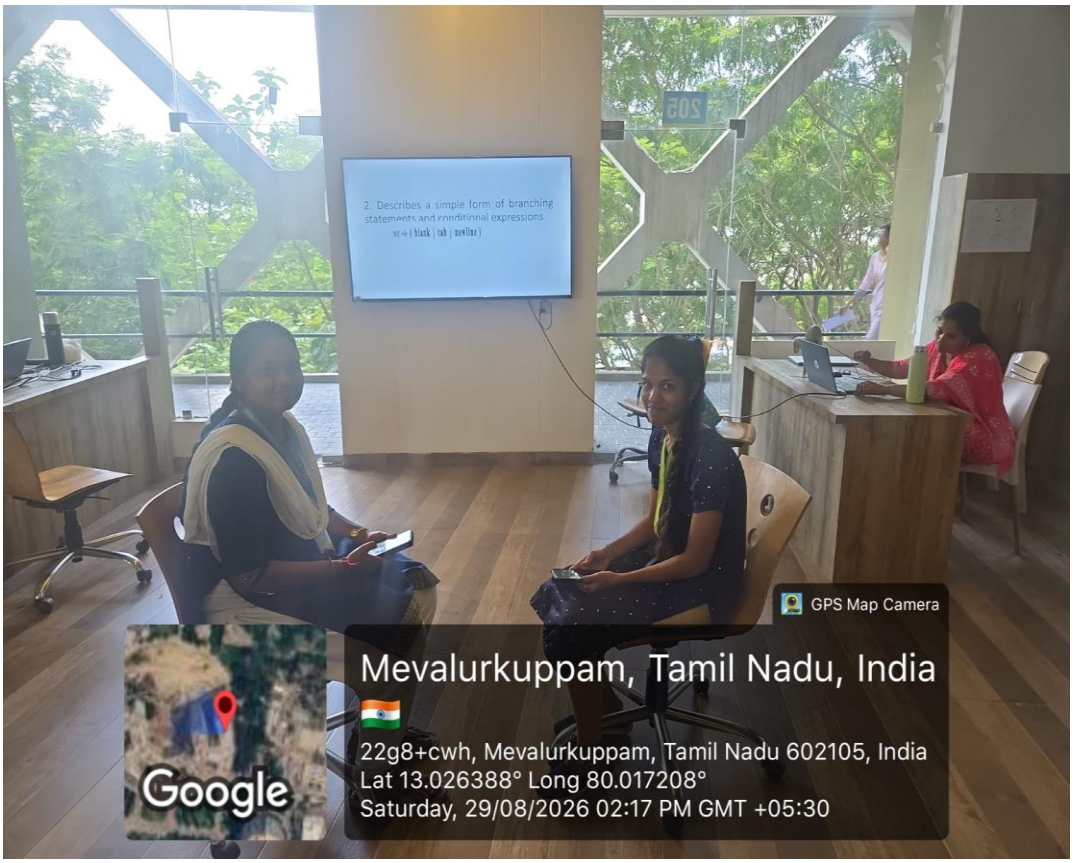
29. What is bottom-up parsing?

It starts with the input and reduces it step-by-step to the start symbol.

30. What is shift-reduce parsing?

It is a bottom-up method that uses two main actions: shift input onto the stack and reduce using grammar rules.

GEO TAG IMAGE:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking was demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately